Project Team:
Safia Khan
Daniel Opemipo Oni

CS 5331/7331 Project Proposal

Objectives/Hypothesis

We will build a classifier that predicts whether a URL is phishing or legitimate based solely on its structural features. We will compare five classification algorithms from Module 4 — Decision Tree, Naive Bayes, Support Vector Machine, k-Nearest Neighbors, and Boosting — on the UCI Phishing Websites dataset, and identify which features drive each model's predictions. The project outputs are a working classifier, a comparative performance analysis, and an interpretation of which features matter most for real-world phishing detection.

We hypothesize that URL-structural and page-level features alone can classify phishing versus legitimate URLs at F1 ≥ 0.90 on a held-out test set, and that boosting will outperform the individual classifiers. We further expect the most predictive features to include domain age (age_of_domain), HTTPS presence (sslfinal_state), and URL length, consistent with Mohammad et al. [2] and Khan et al. [4]. A secondary question we will test, following Bahuang and Beh [3], is whether URL-only features — dropping the five features that require external services such as WHOIS and Google indexing — can retain predictive performance competitive with the full feature set.

Literature Review

1. Mohammad, Thabtah, and McCluskey (2012) [2]

Dataset: The authors, who later created the UCI dataset used in our project, collected 2,500 phishing URLs from the PhishTank archive. They built JavaScript and PHP tools to extract features, organized into four categories: address bar, abnormal request,

HTML/JavaScript, and domain-based features. Some features required querying external services such as WHOIS and Alexa.

Method and findings: Rather than training a classifier, the authors defined precise extraction rules for 17 features (for example, labeling URLs of 54 or more characters as phishing) and measured how often each feature appeared in the phishing sample as an initial indicator of importance. Request URL appeared in every phishing site, followed by Age of Domain (95.6%) and HTTPS/SSL (92.8%), while Disabling Right Click (1.6%) and the “@” symbol (3.6%) were rarest.

Comparison to our work: This paper defines the features we use, but it measures how often features appear among phishing sites only, without comparing them against legitimate sites or testing predictive power. We will measure feature importance directly through trained classifiers. We also test whether their URL-based features can predict phishing without the features that require external lookups.

2. Bahuang and Beh (2022) [3]

Dataset: The authors used the same UCI Phishing Websites dataset (11,055 records, 30 features), but removed five features that depend on third-party services (age of domain, DNS record, page rank, web traffic, and Google index) to speed up feature extraction, leaving 25 features.

Method and findings: They trained Logistic Regression, Naive Bayes, and SVM on an 80/20 train/test split, tuning hyperparameters with grid search. SVM performed best at 95.80% accuracy, followed by Logistic Regression (92.03%) and Naive Bayes (88.91%). Tuning helped Naive Bayes the most, raising its accuracy from 61.06%. The SVM model was then deployed in a Chrome browser extension for real-time detection.

Comparison to our work: This study is the closest to our hypothesis, since it shows that removing third-party features still allows high accuracy. However, it tested only three

classifiers, reported only accuracy for its final comparison, and never compared the reduced feature set against the full set. We extend this by comparing five classifiers, including boosting, on both the full feature set and a URL-based subset, using F1 Score and ROC AUC. Their weak, untuned Naive Bayes result also supports our choice of a categorical Naive Bayes variant suited to this coded data.

3. Khan, Khan, and Hussain (2020) [4]

Dataset: The study compared three phishing datasets, including the UCI dataset used in our project (11,055 websites: 6,157 legitimate and 4,898 phishing). It also used a 10,000-webpage dataset with 48 features and a smaller 1,353-record multi-class dataset.

Method and findings: The authors trained Decision Tree, SVM, Random Forest, Naive Bayes, kNN, and a neural network using 10-fold cross-validation, then repeated the experiments after reducing dimensions with Principal Component Analysis (PCA). On the UCI dataset, Random Forest (95.96%) and the neural network (95.90%) achieved the highest accuracy, while Naive Bayes performed poorly (60.48% accuracy, 28.95% recall). PCA reduced the 30 features to 18 components with only a small drop in accuracy. Its feature ranking placed having_sub_domain and age_of_domain highest.

Comparison to our work: This paper supports our expectation that tree-based models will perform well. Its Naive Bayes result shows that Gaussian Naive Bayes is a poor fit for this coded data. However, PCA creates abstract components that do not correspond to features a browser could actually check. We instead group features by where they come from (the URL itself versus the page or external services), so our results stay interpretable in practical terms. We also include a boosting model, which this study did not test.

4. Almujahid, Haq, and Alshehri (2024) [5]

Dataset: The authors used two datasets: a Mendeley dataset of 10,000 webpages with 48 features, and the same UCI dataset used in our project (11,055 records, 30 features).

Method and findings: On the UCI data, they balanced the classes using SMOTE oversampling and selected 11 features based on their correlation with the label, with SSLfinal_State and URL_of_Anchor scoring highest. They then compared Logistic Regression, kNN, Decision Tree, Random Forest, SVM, XGBoost, a CNN, and a deep learning model with grid search tuning and stratified 10-fold cross-validation. On the UCI data, XGBoost and Random Forest each reached about 98% accuracy after tuning, Decision Tree 97%, SVM 95%, and kNN 94%, while the CNN performed best at 99%.

Comparison to our work: This study tests almost the same set of classifiers we plan to use, so it gives us a direct benchmark. It also supports our hypothesis that boosting will outperform single classifiers. However, it applies SMOTE oversampling, which creates synthetic data points, and selects features statistically. We instead keep the original class distribution with a stratified split, and select features by their source, to test a practical question this study does not address: how much accuracy is lost when only URL-based features are available.

5. Sahingoz, Buber, Demir, and Diri (2019) [6]

Dataset: The authors built their own dataset of 73,575 URLs (36,400 phishing collected from PhishTank, 37,175 legitimate collected from Yandex search results), significantly larger than the UCI dataset we use. They extracted URL-only features — no page content — including word-based NLP features derived from the URL string itself, along with character-distribution and lexical features.

Method and findings: They compared seven classifiers (Decision Tree, AdaBoost, K-Star, kNN, Random Forest, Support Vector Machine, Naive Bayes) using Natural Language Processing features extracted from URLs. Random Forest performed best at

97.98% accuracy. They showed that URL-only features — without needing to load the page or query external services — could achieve high detection accuracy, which directly supports the practical motivation for lightweight real-time filters.

Comparison to our work: This paper supports our secondary hypothesis that URL-only features can achieve strong classification performance. Their dataset is much larger than ours and includes NLP word features we do not use; our contribution is to run the same URL-only versus full-feature comparison within a single dataset (UCI), controlling for the dataset itself, which their study cannot do because they only use URL features.

6. Jain and Gupta (2018) [7]

Dataset: The authors used 2,141 phishing URLs from PhishTank and 1,918 legitimate URLs sampled from Alexa, totaling about 4,000 URLs. They extracted 19 features drawn from both the URL string and the hyperlinks on the loaded page, with a deliberate focus on client-side detection (no reliance on blacklists or external services such as WHOIS).

Method and findings: Their framework ran feature extraction client-side and used Random Forest as the primary classifier, achieving a 99.09% true positive rate with a low false positive rate. They argued that hyperlink-based features extracted at the client offer the best accuracy-latency trade-off for in-browser phishing filters.

Comparison to our work: Their client-side framing is closely aligned with our practical motivation. The main contrast is that their feature set still requires loading the page to extract hyperlinks, whereas our URL-only subset tests whether we can classify before the page loads at all. Our comparison between URL-only and full-feature performance should clarify whether the extra page-load cost is justified by meaningful accuracy gains.

7. Ma, Saul, Savage, and Voelker (2009) [8]

Dataset: The authors collected a labeled corpus of roughly 34,000 URLs (benign and malicious) over several months from live web feeds, blacklists, and user reports. Features combined lexical properties of the URL string with host-based features drawn from DNS records, WHOIS lookups, geographic information, and existing blacklist membership.

Method and findings: They compared Naive Bayes, Support Vector Machines, and Logistic Regression on the task of classifying URLs as benign or malicious without visiting the pages. Their work demonstrated that supervised learning on lexical and host-based features could reach 95-99% accuracy, outperforming blacklist-only defenses on previously unseen URLs. This paper is widely cited as foundational evidence that URL classification is viable without rendering the page.

Comparison to our work: This is the foundational paper establishing the premise behind our project. Their feature mix spans URL text and third-party-lookup features, which parallels the full-versus-URL-only split we investigate. Our contribution is a controlled comparison on a modern, publicly reproducible benchmark (UCI) with a wider classifier lineup, adding Decision Tree, kNN, and Boosting beyond their original three.

8. Rao and Pais (2019) [9]

Dataset: The authors assembled a dataset of 7,900 URLs (3,450 phishing from PhishTank, 4,450 legitimate from Alexa). They extracted 20 features drawn from three sources: the URL string, the hyperlinks on the page, and third-party services (WHOIS, Alexa, Google Index).

Method and findings: They compared Random Forest, oblique Random Forest, AdaBoost, Logistic Regression, Decision Tree, Naive Bayes, Support Vector Machine, and k-Nearest Neighbors. Random Forest and oblique Random Forest achieved the

highest accuracy at 99.31%. Their feature analysis grouped predictors by source (URL, page, third-party), which is the same practical taxonomy we use.

Comparison to our work: Their grouping of features by source aligns directly with our hypothesis. However, they do not systematically compare performance across those feature-source groupings — they only report overall accuracy on the full set. Our URL-only versus full-feature comparison tests the specific trade-off between feature cost and detection accuracy that their framework assumes but does not measure.

9. Chiew, Tan, Wong, Yong, and Tiong (2019) [10]

Dataset: The authors evaluated their method on the same UCI Phishing Websites dataset we use (11,055 records, 30 features), plus a second phishing dataset with 48 features. They did not add new data; the focus of the work was on feature selection.

Method and findings: They proposed the Hybrid Ensemble Feature Selection (HEFS) framework, which combines a cumulative distribution function gradient method with a data perturbation ensemble to identify the most important features. On the UCI dataset, HEFS reduced the feature set from 30 to around 10 while Random Forest still achieved roughly 96% F1-score, showing that most of the information in the 30 features is concentrated in a small core.

Comparison to our work: This paper is useful as a direct benchmark on the same dataset. It suggests that aggressive feature selection does not sharply hurt performance, which strengthens the plausibility of our URL-only subset experiment. Our contribution differs in that we select features by their source and interpretability (features a browser can check without external calls), rather than selecting them statistically to maximize accuracy.

Dataset

Source & Access

We will use the Phishing Websites dataset from the UCI Machine Learning Repository, created by Rami Mohammad and Lee McCluskey [1] and introduced in Mohammad, Thabtah, and McCluskey [2]. The dataset is publicly available under a Creative Commons Attribution 4.0 (CC BY 4.0) license at: https://archive.ics.uci.edu/dataset/327/phishing+websites

We access it directly in Python with the ucimlrepo package (fetch_ucirepo(id=327)), which returns the features and labels as pandas DataFrames. A copy is saved to our shared project folder so both team members work from the identical file.

Domain & Context

The data comes from cybersecurity, specifically the detection of phishing websites: fraudulent sites that imitate legitimate organizations to steal personal information such as login credentials and credit card numbers. The websites were collected mainly from the PhishTank and MillerSmiles phishing archives and from Google search results. Each record is one website, and each feature is a rule-based indicator of whether a property of that site appears legitimate, suspicious, or phishing.

Size & Structure

The dataset contains 11,055 records and 31 columns: 30 input features and 1 class label. All features are categorical and integer-encoded:

21 binary features coded {−1, 1}
8 three-level features coded {−1, 0, 1}: url_length, having_sub_domain, sslfinal_state, url_of_anchor, links_in_tags, sfh, web_traffic, links_pointing_to_page
1 binary feature coded {0, 1}: redirect

In general, −1 indicates a phishing-like value, 0 a suspicious value, and 1 a legitimate-looking value. The features fall into four broad groups:

characteristics of the URL itself (e.g., URL length, use of an IP address, the “@” symbol, number of subdomains)
abnormal requests and links on the page (e.g., anchor URLs, form handlers)
HTML and JavaScript behavior (e.g., pop-up windows, iframes, disabled right-click)
domain-level properties (e.g., domain age, DNS record, web traffic, Google indexing)

Target Variable

The target variable is result, a binary label where −1 = phishing and 1 = legitimate.

Class	Label	Count	Percentage
Legitimate	1	6,157	55.7%
Phishing	-1	4,898	44.3%
Total		11,055	100%

Known Issues

- Missing values: None. We verified there are 0 missing values across all 31 columns, so no imputation is needed.
- Class imbalance: The classes are mildly imbalanced (55.7% vs. 44.3%). We will use a stratified train/test split to preserve this ratio, and report precision, recall, F1-score, and ROC-AUC alongside accuracy so performance on the phishing class is not overstated.
- Pre-discretized features: The features were already converted from raw measurements into coded categories (e.g., URL length is recorded as short, medium, or long rather than a character count). This loses some detail and means feature scaling will have limited effect. It also affects model choice: for Naive Bayes, a Categorical or Bernoulli model fits this data better than Gaussian Naive Bayes, which assumes continuous values.

- Age of the data: The data was collected before its 2015 donation to UCI, so phishing tactics have likely changed since then (e.g., phishing sites now commonly use HTTPS). Our findings describe this dataset and may not fully reflect present-day phishing.

Relevance to Objectives

This dataset suits our project because it provides a large, labeled, real-world sample of phishing and legitimate websites, with features already extracted, so we can focus on comparing classification methods and analyzing which features drive predictions. Its features span different sources of evidence, from properties visible in the URL alone to properties that require loading the page or querying external services. That lets us test how well URL-based features alone perform compared to the full feature set. The dataset has also been widely used in prior research, so we can compare our results directly with published findings.

References

[1] R. Mohammad and L. McCluskey, “Phishing Websites,” UCI Machine Learning Repository, 2012. [Online]. Available: https://doi.org/10.24432/C51W2X

[2] R. M. Mohammad, F. Thabtah, and L. McCluskey, “An assessment of features related to phishing websites using an automated technique,” in Proc. Int. Conf. Internet Technology and Secured Transactions (ICITST), London, U.K., 2012.

[3] R. Bahuang and D. M. Y. Beh, “Detecting phishing uniform resource locator (URL) using machine learning,” J. Computing Technologies and Creative Content, vol. 7, no. 2, pp. 35–41, Dec. 2022.

[4] S. A. Khan, W. Khan, and A. Hussain, “Phishing attacks and websites classification using machine learning and multiple datasets (a comparative analysis),” 2020, arXiv:2101.02552.

[5] N. F. Almujahid, M. A. Haq, and M. Alshehri, “Comparative evaluation of machine learning algorithms for phishing site detection,” PeerJ Computer Science, vol. 10, Art. no. e2131, 2024, doi: 10.7717/peerj-cs.2131.

[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Systems with Applications, vol. 117, pp. 345–357, 2019.

[7] A. K. Jain and B. B. Gupta, "Towards detection of phishing websites on client-side using machine learning based approach," Telecommunication Systems, vol. 68, no. 4, pp. 687–700, 2018.

[8] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: learning to detect malicious web sites from suspicious URLs," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), Paris, France, 2009, pp. 1245–1254.

[9] R. S. Rao and A. R. Pais, "Detection of phishing websites using an efficient feature-based machine learning framework," Neural Computing and Applications, vol. 31, no. 8, pp. 3851–3873, 2019.

[10] K. L. Chiew, C. L. Tan, K. Wong, K. S. C. Yong, and W. K. Tiong, "A new hybrid ensemble feature selection framework for machine learning-based phishing detection system," Information Sciences, vol. 484, pp. 153–166, 2019.